

Step 3 — Add the Resource and the Prompt — Hints

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 3 depth levels: **Basic** (the plain idea), **Intermediate** (proper Python), **Advanced** (why the Tool/Resource/Prompt line actually matters in a real client). Read Basic first even if you already know the `mcp` SDK's syntax — it's the fastest way to spot exactly what each deeper level adds.

- [Hint 1 — The idea, and the exact pieces](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The idea, and the exact pieces

Basic Version

This step isn't really about new syntax — `@mcp.resource(...)` and `@mcp.prompt()` look almost identical to `@mcp.tool()`, which you already know from Steps 1 and 2. It's about a decision: for each of these two new things, ask "does calling this *change* anything?" `notes://all` doesn't — it only reads. `summarize_notes` doesn't either — it only hands back a template string, it never calls a model itself. That's why neither one is a third or fourth Tool.

Things to use: - `@mcp.resource("notes://all")` — decorates a function with no arguments that returns a string. The string inside the decorator is the Resource's URI, not a file path — you're making it up, following the `scheme://path` shape. - `@mcp.prompt()` — decorates a function whose parameters become the Prompt's arguments, exactly like a Tool's parameters do, and whose return value is the prompt text itself.

Don't reach for a database write inside either of these — if you find yourself wanting one, that's a sign the thing you're building should have been a Tool instead.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

The real shape:

```
@mcp.resource("notes://all")
def all_notes() -> str:

@mcp.prompt()
def summarize_notes(start_date: str, end_date: str) -> str:
```

`all_notes()` needs a new `db.py` function, `list_all_notes(path) -> list[tuple[int, str, str]]` (`id`, `text`, `created_at`) — a plain `SELECT id, text, created_at FROM notes ORDER BY id`. The Resource function itself just formats that list into one readable string — something like one line per note, `"[{id}] {text} ({created_at})"`, joined with newlines. If there are no notes yet, return a plain sentence saying so ("No notes yet."), not an empty string that looks like something broke.

`summarize_notes()` takes `start_date` and `end_date` as plain strings (keep the date format simple — `"YYYY-MM-DD"` — this project doesn't need real date parsing, just two strings dropped into a template) and returns an f-string prompt, something in the shape of: *"Summarize the notes created between {start_date} and {end_date} in 2-3 sentences, grouped by theme if there's more than one."* Nothing in this function touches the database or calls a model — it only builds and returns text.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Think about why a real MCP client (Claude Desktop, an IDE) treats these three kinds of things differently, not just as a naming convention.

Tools usually require the user's explicit approval before a client will actually run them, every time, because they can *do* something — write data, send something, spend money. **Resources** are commonly listed and attached to a conversation directly, the same way a user might drag a file into a chat — no approval dialog needed, because reading `notes://all` can't change anything no matter how many times it's read. **Prompts** typically show up as a menu of ready-made starting points a user can pick from, filling in the parameters through a form the client builds from your function's signature — the same way `add_note`'s single `text: str` parameter becomes a form field for a Tool.

If `notes://all` had been built as a third Tool instead (`list_notes() -> str`, no arguments), a real client would likely still ask the user to approve running it, every single time, for something that's provably harmless to call as many times as anyone likes. That's not just clumsy — it trains users to click "approve" reflexively, which is exactly the habit that makes an *actual* dangerous Tool call more likely to get rubber-stamped without a second look. Choosing Resource over Tool here isn't a style preference; it's picking the classification that matches the real risk, so approval friction stays reserved for things that actually deserve it.

For the Prompt, imagine a teammate joining this project later, who doesn't know your preferred phrasing for "summarize notes in a date range." If that phrasing only exists typed by hand into whichever chat client you personally use, they have to reconstruct it from scratch, and probably word it slightly differently. Because it's a Prompt the *server* provides, every client that connects gets the exact same, already-tested wording, discovered the same way `list_tools()` discovers Tools — this is server-side prompt engineering, shared once instead of re-typed by every person and every client that needs it.

Difference between Basic, Intermediate, and Advanced: Basic and Intermediate get the Resource and Prompt working correctly. Advanced explains the actual mechanism that makes the Tool/Resource/Prompt choice matter in a real client — approval friction, drag-and-drop context, and a shared prompt menu — not just "the docs say to pick one of three categories."

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
db.py:
    list_all_notes(path): select every note, ordered by id

server.py:
    resource notes://all:
        get every note, format one per line
        if there are none, say "No notes yet."

    prompt summarize_notes(start_date, end_date):
        return a template string asking to summarize notes
        created between start_date and end_date
```

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

```
# db.py (new function)
def list_all_notes(path: str) -> list[tuple[int, str, str]]:
    conn = sqlite3.connect(path)
    rows = conn.execute("SELECT id, text, created_at FROM notes ORDER BY id").fetchall()
    conn.close()
    return rows
```

```
# server.py (new pieces)
from db import list_all_notes

@mcp.resource("notes://all")
def all_notes() -> str:
    """A read-only listing of every note currently stored."""
    rows = list_all_notes(DB_PATH)
    if not rows:
        return "No notes yet."
    lines = [f"[{row[0]}] {row[1]} ({row[2]})" for row in rows]
    return "\n".join(lines)

@mcp.prompt()
def summarize_notes(start_date: str, end_date: str) -> str:
    """A reusable template for summarizing notes created in a date range."""
    return (
        f"Summarize the notes created between {start_date} and {end_date} "
        "in 2-3 sentences. Group related notes by theme if there's more than one."
    )
```

Write this yourself, run it through the Inspector's Resources and Prompts tabs, and confirm both against the [Solution](#).

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

The Intermediate version already works. The one genuine design decision left is date filtering: should `summarize_notes` only ever return the *template*, no matter what dates are passed in (what's shown above), or should the server also filter `list_all_notes()`'s rows down to that date range and somehow include them? Sketch both directions yourself before checking the Solution — there's a real, defensible reason to prefer the simpler one, and seeing why is worth more here than being handed the answer.

```
option A: summarize_notes returns ONLY the instruction text
- the client is expected to attach notes://all (or a filtered
  version of it) to the conversation itself, alongside this prompt
```

```
option B: summarize_notes also fetches and embeds the matching notes'
  text directly inside the returned string
```

```
which one keeps "Prompt" and "Resource" doing separate, single jobs,
and which one starts blurring the two together?
```

Difference between Basic, Intermediate, and Advanced: Basic and Intermediate build a correct Resource and Prompt. Advanced is a genuine design fork worth thinking through yourself — whether a Prompt should ever reach into the database at all, or stay a pure template and leave data-fetching to the Resource it's meant to be paired with.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)